

```
//
=====
=====
// COPYRIGHT (C) 2026 [Samuel Fearnley]. ALL RIGHTS RESERVED.
// This shader is provided for use in projects but may not be resold or
// redistributed as source code without express permission.
//
=====
=====
// "RenderPipeline" = "UniversalPipeline":
// This is a "lock and key" tag. It tells Unity, "Only run this shader if
the project is using URP."
// If you tried to use this in the old Built-in pipeline, it WON'T run.
//
=====
=====

=====
=====
VHS MASTER SHADER: THE CODE BIBLE (REVISED)
=====
=====

[GLOBAL TIME & STABILIZATION]
-----
-----
- float t_stable = frac(_Time.y);
- REASON: I use 'frac' to strip the whole seconds and keep a looping 0-
to-1
decimal. This prevents "Floating Point Precision" errors. If the game
runs for hours, raw _Time.y becomes a massive number that makes noise
jitter or stop; t_stable keeps the math "calm" and precise.

---

[MODULE: GLOBAL DISTANCE]
-----
-----
- float dist = dot(centeredUV, centeredUV);
- REASON: I calculate this at the very top of my logic.
- PURPOSE: Since Fisheye, Vignette, Edge Blur, and Chromatic Aberration
all
rely on "How far is this pixel from the center?", calculating it once
saves performance and ensures that the colors split even if the
Fisheye toggle is turned off.

---

[1. COORDINATE SPACES]
-----
-----
- input.texcoord (UV): The original 0-to-1 map of the screen.
- distortedUV: A mutable copy of the UVs.
- REASON: I create this copy so I can "bend" or "shake" the pixels
```

without losing the original reference of where the screen center is.

[2. FISHEYE DISTORTION (LENS BULGE)]

- FORMULA: $\text{distortedUV} = 0.5 + (\text{centeredUV} * \text{_Zoom}) * (1.0 + \text{_Strength} * \text{pow}(\text{dist}, \text{_Power}))$
- REASON: I use the 'pow' (Power) function here to "protect" the middle. Linear distortion bends everything; Exponential distortion (Power) keeps the center flat and sharp while forcing the extreme warping to only happen at the corners.
- OPTIMIZATION: I use 'dot(uv, uv)' to find distance because it avoids the square root (sqrt) operation, saving GPU cycles.

[3. CHROMATIC ABERRATION (RADIAL SPLIT)]

- FORMULA: $\text{offset} = \text{abbDir} * (\text{_Intensity} * \text{pow}(\text{dist}, 2.0))$
- REASON: I am making the color-split "Dynamic." By multiplying the intensity by the distance squared, the Red/Blue channels only begin to separate as they move away from the center.
- LOOK: This ensures the center of the screen (where the player looks) is clear, while the edges look like a cheap, refracting lens.
- FORMULA: $\text{r_uv} += \text{abbDir} * (\text{_AbbIntensity} * \text{pow}(\text{dist}, 2.0));$
- REASON: I use a radial direction (center to pixel) combined with an exponential falloff.
- RESULT: The center of the lens is perfectly focused. The edges "shatter" into rainbows.
- SYNTAX: I use 'pow(dist, 2.0)' to make the transition from the sharp center to the blurry edges feel smooth and professional.

[TOPIC: THE SCIENCE OF THE SPLIT]

- DEFINITION: "Lateral Chromatic Aberration" occurs when a lens magnifies colors differently.
- REASON: In VHS effects, we use this to simulate "low-quality tracking" or "cheap glass" look. It makes the image feel less "perfectly digital."
- FORMULA: $\text{distortedUV} = \text{center} + (\text{direction} * (1.0 + \text{amount} * \text{distance}))$
(We move the Red UV 'out' and the Blue UV 'in' to create the gap).

[4. VIGNETTE (OPTICAL FALLOFF)]

- FORMULA: $\text{color.rgb} *= \text{saturate}(1.0 - \text{pow}(\text{vDistSquared} * \text{_Strength}, \text{_Size}))$
- REASON: I simulate the physical thickness of a lens blocking light at the edges.
- SYNTAX: I use 'saturate' to clamp values between 0 and 1. This is my

safety check to ensure that high darkness settings don't result in "negative light" which would cause neon pixel glitches.

[5. TRACKING GLITCH & VERTICAL JUMP]

- FORMULA: $\text{step}(\text{Threshold}, \sin((\text{uv.y} * \text{Amount}) - (\text{Time} * \text{Speed})))$
- REASON: I use a Sine wave to create scrolling bands. The 'step' function turns that wave into a "Binary" mask (Either 0 or 1).
- RESULT: This creates the sharp-edged horizontal bars that shift the image left or right, mimicking a VCR "tracking" error.

[6. RADIAL EDGE BLUR]

- FORMULA: $\text{color} = (\text{sample1} + \text{sample2} + \text{sample3} + \text{sample4}) / 4$
- REASON: A GPU cannot "blur" in one go. I have to take 4 separate "peeks" (samples) at the texture with slight offsets and average them. By tying the offset to 'dist', the blur only activates at the screen edges.

[7. COLOR BLEED (MAGNETIC SMEAR)]

- FORMULA: $\text{max}(\text{OriginalColor}, \text{NeighborColor})$
- REASON: Old analog signals had low "Chroma" (color) resolution. By sampling a neighbor and using 'max', I make bright colors "smear" to the right, simulating the magnetic lag of VHS tape.

[TOPIC: BLENDING MODES]

- DEFINITION:
 - $\text{lerp}(a, b, t)$: A linear blend between two colors.
 - $\text{max}(a, b)$: Only picks the brightest value.
- REASON: We use lerp for VHS Bleed because analog signals don't 'compare' brightness; they simply fail to update fast enough, causing the previous pixel's color to spill into the next one.
- FORMULA: $\text{color} = (1.0 - t) * a + (t * b)$

[8. SCANLINES & NOISE]

- FORMULA: $\sin(\text{uv.y} * \text{Density})$ for lines; $\text{dot}(\text{uv}, \text{Seed})$ for noise.
- REASON: Scanlines simulate the physical phosphor grid of a CRT TV. Noise provides the "Grain" that makes the digital image feel like an analog chemical/magnetic medium.

=====

=====